

Veridian

自托管控制面：精简版 Supabase 开源组件 Self-Hosted Control Plane – a Minimal Supabase OSS Subset

只用 Postgres + GoTrue + PostgREST • 管理员专属 Studio • 兼容官方 supabase-js SDK

本文档是《[readme/workspace-sync/design.tex](#)》控制面章节的补充：
研究 Supabase 实际开源架构后，给出”自托管而非用 Supabase Cloud”的具体落地方案，
对应可运行的部署文件位于仓库 `control-plane/` 目录。

Contents

1	研究结论：Supabase 完整自托管栈有多重	2
2	精简方案：三个真实服务 + 一个反代	3
2.1	为什么可以去掉 Kong	3
2.2	最终架构图	3
3	管理员专属访问：Studio 网络隔离	3
4	取舍：不跑 Realtime 服务	4
5	控制面的核心用途：工作空间 → GitHub 仓库映射	4
6	部署文件清单	4
7	待办：客户端集成（下一步）	4

1 研究结论：Supabase 完整自托管栈有多重

Supabase 官方自托管方案（`docker-compose.yml`，来自 [supabase/supabase](https://github.com/supabase/supabase) 仓库的 `docker/` 目录）由 12–13 个容器化服务组成：

服务	技术栈	Veridian 是否需要
Postgres	C	✓需要（唯一的数据存储）
GoTrue (Auth)	Go	✓需要（登录/注册/邀请接受）
PostgREST	Haskell	✓需要（自动生成的 REST API）
Kong	Lua/OpenResty	× 不需要——见 §2.2
Realtime	Elixir/Phoenix	× 不需要——见 §4
Storage API	Node.js	× 不需要，文件走 GitHub 仓库/云盘
postgres-meta	Node.js	仅 Studio 依赖它，管理员按需启用
imgproxy	Go	× 完全不需要
Edge Functions	Deno	× 完全不需要
Studio	Next.js	仅管理员本机/隧道访问
Analytics/Vector	Elixir/Rust	× 完全不需要

为什么不能整套塞进 Electron 客户端

这 12–13 个服务的设计前提是“跑在一台服务器上”：需要 Docker、持久化卷、常驻进程、几 GB 内存。要求每一个 Veridian 终端用户的电脑上都装 Docker 并常驻运行这一整套栈，对个人桌面应用是不可接受的——这与“轻量 Electron 应用、零后台服务依赖”的既有定位（见 `readme/analysis/comparison.tex`）直接冲突。

真正可行的路径

控制面本质上需要一个**管理员运营的共享服务**——这是“多人共享同一份成员/角色记录”这件事本身决定的，无法绕开。真正的问题不是“要不要有服务器”，而是“这个服务器需要跑多重”。精简到 Postgres + GoTrue + PostgREST 三个真实服务后，管理员可以在一台免费/廉价的 VPS 上轻松承担，而不需要理解或运维 Kong/Realtime/Storage/Edge Functions 这些 Veridian 用不上的部分。

Sources:

- [supabase/supabase docker/README.md](https://github.com/supabase/supabase/blob/master/docker/README.md)
- [Supabase Docs – Self-Hosting with Docker](#)
- [supabase/auth \(GoTrue\)](#)
- [PostgREST](#)

2 精简方案：三个真实服务 + 一个反代

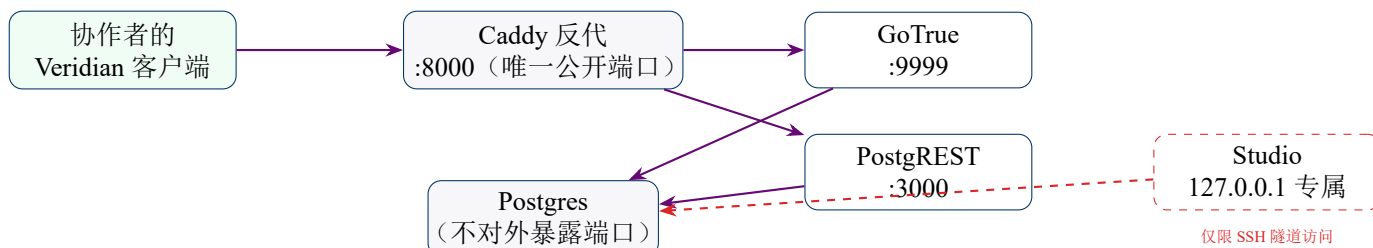
2.1 为什么可以去掉 Kong

Kong 在完整栈里做两件事：统一路径路由，以及给浏览器端跨域请求做 CORS 处理。Veridian 的控制面调用发生在 **Electron** 主进程里（就像现有的 CrossRef/MinerU API 调用一样），而不是渲染进程——主进程没有 **CORS** 限制，这是浏览器沙箱层面的安全机制，Node.js 的 `fetch` 完全不受影响。JWT 校验本身也不需要网关来做：PostgREST 用 `PGRST_JWT_SECRET` 自行验证请求头里的 JWT。

Kong 唯一还有价值的地方是”让客户端只需要记一个 base URL”——这个用一个十几行的 Caddy 反代（`control-plane/Caddyfile`）就能达到同样效果，换来的好处是客户端仍可以用官方 `@supabase/supabase-js` SDK（免去自己实现 JWT 刷新/重试逻辑）。

```
:8000 {
  handle /auth/v1/* { uri strip_prefix /auth/v1; reverse_proxy gotrue:9999 }
  handle /rest/v1/* { uri strip_prefix /rest/v1; reverse_proxy postgrest:3000 }
}
handle { respond "Veridian control plane -- not a public endpoint" 404 }
```

2.2 最终架构图



3 管理员专属访问：Studio 网络隔离

对应你提出的”只有管理员账号可以查看控制面”这一要求，具体实现不是应用层的账号判断（那样仍然暴露了整张表可被查询的攻击面），而是 **网络层隔离**：

```
studio:
  ports:
    - "127.0.0.1:3001:3000" # 只绑定回环地址，公网完全不可达
```

管理员需要浏览原始表时，通过 SSH 隧道把远程的 `127.0.0.1:3001` 映射到自己电脑：

```
ssh -L 3001:127.0.0.1:3001 admin@your-server
# 然后在自己电脑打开 http://localhost:3001
```

普通协作者的 Veridian 客户端从未、也不可能访问到 Studio——它们只知道 `proxy:8000` 这一个地址，而这个地址背后只有 Auth 和 REST 两类受 RLS 保护的接口，看不到任何”管理后台”。

4 取舍：不跑 Realtime 服务

design.tex 早期版本设想用 Supabase Realtime 推送成员/角色变更。精简为三服务后，这项能力被有意去掉：

轮询取代推送，够用即可

成员/角色变更是低频事件（一个工作空间一个月可能就邀请几次人），不需要毫秒级传播。客户端在以下时机主动查询 workspace_members：应用启动时、每次同步 push/pull 前、以及用户手动点”刷新”——权限变更几分钟内生效即可，不追求”实时”。这与 Zotero 自身的成员/ 权限模型（也非实时）一致。

5 控制面的核心用途：工作空间 → GitHub 仓库映射

按你的要求，控制面**不是**一个通用的 Notion 式内容数据库——它只做一件事：记住”这个工作空间的数据在哪个 GitHub 仓库（或云盘路径）里，谁能以什么角色访问它”。control-plane/schema.sql 里 workspaces 表的核心字段正是：

```
1 sync_backend_type    sync_backend not null,           -- 'git' / 'cloud_folder'
2 sync_backend_config jsonb not null default '{}',      -- 仓库地址 / 云盘路径
```

角色（owner/admin/editor/viewer）判断的**唯一后果**就是：该成员的 Veridian 客户端是否被允许对这个 sync_backend_config 指向的 GitHub 仓库发起 push（写）——控制面不存储、不理解文献内容本身，那部分完全在 §design.tex §4 描述的数据面（GitHub 仓库）里。

6 部署文件清单

文件	内容
control-plane/docs/PostgresSetup.md	PostgreSQL + PostgREST + Caddy + 管理员专属 Studio/postgres-meta
control-plane/Caddyfile	本地路径转发配置
control-plane/schema.sql	角色权限初始化 + auth.uid() 等辅助函数 + workspaces/workspace_members/invites/devices 四张表 + RLS 策略
control-plane/.env.example	密钥模板（真实 .env 已被仓库根 .gitignore 拦截，不会被提交）
control-plane/README.md	部署步骤、管理员访问方式、与工作空间/GitHub 仓库关系的说明

7 待办：客户端集成（下一步）

本文档与 control-plane/ 目录只完成了”控制面服务本身”。Veridian 客户端侧仍需实现（对应 design.tex §6 模块清单）：

- main/services/PermissionSource.ts 的 Supabase 实现（封装 @supabase/supabase-js，指向 proxy:8000）
- main/services/WorkspaceService.ts：创建工作空间时选择 GitHub 仓库或云盘路径、写入 sync_backend_config

- 邀请/成员管理 UI (`InviteDialog.tsx/MembersPanel.tsx`)
- 设置界面里新增”连接到控制面”的一栏（填入管理员提供的 `SITE_URL`）

尚未开始编码——这是下一步需要确认的实施顺序。